The Cost of Senders for Coroutine I/O

Document Number:	D4123R0
Date:	2026-03-17
Audience:	LEWG, SG1

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

References

Abstract

Every concession granted. The gap remains. I/O users pay for what they do not need.

This paper grants P3552R3[1] every engineering fix that has been proposed or discussed, assumes they all ship, and compares against the best possible conforming implementation of `std::execution::task` - one that eliminates every cost not mandated by [exec.task]. The gap is in the task-to-task suspend path. It is spec-mandated.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version

---

1. Disclosure

The author developed and maintains Corosio[4] and Capy[5] and believes coroutine-native I/O is the correct foundation for networking in C++. The author provides information, asks nothing, and serves at the pleasure of the chair.

The author regards `std::execution` as an important contribution to C++ and supports its standardization for the domains it serves well - GPU dispatch, heterogeneous execution, and compile-time work-graph composition among them. Nothing in this paper argues for removing, delaying, or diminishing `std::execution`.

---

2. The Concessions

This paper grants `std::execution::task` every engineering fix that has been proposed, discussed, or implied. The following are assumed to ship:

- I/O operations return awaitables, not senders. The template operation state problem (P4088R0[6] Section 6.1) does not arise.
- Symmetric transfer works task-to-task. The stack overflow vulnerability (P3801R0[7]) is resolved.
- Frame allocator timing is fixed. The rework in D3980R0[8] ships. The allocator reaches `promise_type::operator new` before the frame is allocated.
- `AS-EXCEPT-PTR` does not convert routine `error_code` to `exception_ptr`. Routine I/O errors do not become exceptions.
- Compound results are handled inside the coroutine body via structured bindings. `auto [ec, n] = co_await sock.read_some(buf)` works.
- `co_yield with_error` is unnecessary. Compound results stay in the coroutine body. The mechanism that P3801R0[7] identified as blocking symmetric transfer is not needed.
- `IoEnv` is standardized as the networking environment, carrying a type-erased executor and a stop token.
- The promise delivers `IoEnv` to I/O awaitables through `await_transform`.
- Type-erased streams work under both models. Zero per-operation allocation.
- Separate compilation works under both models. I/O functions go in `.cpp` files.
- ABI stability works under both models. The vtable layout does not change.
- `when_all` and `when_any` provide structured concurrency under both models. Children complete before the parent resumes. Stop tokens propagate.
- Predicate-based combinators are achievable under both models. A custom `when_all` that inspects `set_value` arguments through a predicate can cancel siblings on I/O errors. Both models can build this.
- No performance gap on the I/O hot path. The awaitable, the syscall, the resumption, and the frame allocation are identical.
- Cross-domain bridges work. P4092R0[9] and P4093R0[10] demonstrate sender-to-coroutine and coroutine-to-sender interoperation.
- Compound I/O results are routed through `set_error(tuple(ec, T...))` on failure and `set_value(T...)` on success. The standard `when_all` works correctly with this routing. Both values are preserved in the error payload.
- The reference implementation of P3552R3[1] is a work in progress. It introduces costs beyond what [exec.task] mandates. This paper does not compare against any existing implementation. It compares against the best possible conforming implementation - one that eliminates every cost not mandated by the normative text of [exec.task].

Everything above is granted without reservation.

---

3. Two Paths

Both models require an I/O environment - a bundle of state that every I/O operation needs: a type-erased executor (to submit work to the reactor), a stop token (for cancellation), and optionally a frame allocator. P4003R0[16] defines this as `io_env` and specifies the `IoAwaitable` concept that consumes it. In the coroutine-native model, the promise carries `io_env` directly. In the sender model, the `Environment` template parameter of `std::execution::task` serves this role; this paper uses `IoEnv` to denote an `Environment` specialized for I/O.

The same I/O operation. The same user code. Two task types.

3.1 Path A: Coroutine-Native Task

```
io::task<void> session(tcp_socket& sock)
{
    char buf[1024];
    auto [ec, n] = co_await sock.read_some(
        mutable_buffer(buf, sizeof buf));
    if (ec)
        co_return;
    co_await process(buf, n);
}
```

One template parameter. The promise carries `io_env` directly - a type-erased executor, a stop token, and an optional frame allocator. Constructed once at task start. Every I/O operation receives a pointer to it.

When `session` does `co_await process(buf, n)` - a child task - the awaiter stores the continuation handle and returns the child's handle. Symmetric transfer. Two pointer stores.

3.2 Path B: `std::execution::task<T, IoEnv>`

```
std::execution::task<void, IoEnv>
session(tcp_socket& sock)
{
    char buf[1024];
    auto [ec, n] = co_await sock.read_some(
        mutable_buffer(buf, sizeof buf));
    if (ec)
        co_return;
    co_await process(buf, n);
}
```

Two template parameters. When `session` does `co_await process(buf, n)`, the C++26 specification requires a sequence of operations that do not exist in Path A.

Spec-mandated costs (per task-to-task `co_await`)

The `co_await` goes through `await_transform` ([task.promise] paragraph 9), which calls `as_awaitable`. The awaiter calls `connect` ([task.members] paragraph 3-4), which creates a `state<Rcvr>` object. The `state` constructor ([task.state] paragraph 2) must:

1. Store the coroutine handle and the receiver.
2. Construct `own-env` from `get_env(rcvr)`.
3. Construct `Environment` from `own-env` (or from `get_env(rcvr)`, or default).

Then `state::start` ([task.state] paragraph 4) must:

4. Initialize `SCHED(prom)` with `scheduler_type(get_scheduler(get_env(rcvr)))` - extract the scheduler from the receiver's environment.
5. Set up stop token bridging: initialize `prom.token` and `prom.source` such that stop requests from the parent propagate to the child.
6. Call `handle.resume()`.

The promise also carries an `error-variant` ([task.promise] paragraph 2) - a `variant<monostate, Es..., exception_ptr>` with all possible error types. This is constructed once per promise, not per `co_await`, but it is present in every sender-aware task and absent from the coroutine-native task.

These costs are normative. No conforming implementation can eliminate them.

These costs are normative. No conforming implementation can eliminate them. The analysis that follows compares Capy's `task<T>` against the best possible conforming `task<T, IoEnv>`. All implementation-quality costs beyond the spec - virtual dispatch, scheduler comparison on resume, reschedule cycles - are granted as concessions (Section 2).

---

4. The Gap

The table below shows the spec-mandated costs that exist in `task<T, IoEnv>` but not in the coroutine-native `task<T>`. These are irreducible: no conforming implementation can eliminate them. All implementation-quality costs are granted as concessions (Section 2).

Property	Coroutine-native `task<T>`	Best `task<T, IoEnv>`
Template parameters	1	2
Sender concept instantiation per task	0	1 per task type in chain
Task-to-task `co_await` suspend	2 pointer stores	`state<Rcvr>` construction + scheduler extraction
Task-to-task `co_await` stop token setup	0 (already in `io_env*`)	Bridge parent token to `prom.source`
Error storage in promise	Bare `exception_ptr`	`variant<monostate, Es..., exception_ptr>`

---

5. The Gap Explained

5.1 Template Parameters

`task<T>` vs `task<T, IoEnv>`. Two spellings. A function returning one cannot be assigned to a variable of the other. Users must know which to use. Library interfaces must choose. Asio has lived with this for `awaitable<T, Executor>` for years. It is friction, not a structural problem.

5.2 Sender Concept Instantiation

Every `task<T, IoEnv>` satisfies the `sender` concept ([task.class] paragraph 1). The spec mandates `completion_signatures`, a `connect` member function returning `state<Rcvr>`, and nested type aliases for `allocator_type`, `scheduler_type`, `stop_source_type`, `stop_token_type`, and `error_types`. These are present in every task type.

In a five-coroutine chain, five task types are instantiated, each carrying this machinery. None of the intermediate coroutines use it - they pass results via `co_await`, not via `connect` / `start`.

In the coroutine-native model, zero sender instantiations occur inside the chain. One bridge at the edge (P4093R0[10]) connects the chain to the sender world.

5.3 Task-to-Task `co_await` Cost

In the coroutine-native model, `co_await child_task` calls `await_suspend`, which stores the continuation handle and the `io_env` pointer, then returns the child's handle. Two pointer stores. Symmetric transfer.

In the sender model - even in the best possible conforming implementation - `co_await child_task` must perform the following operations mandated by [exec.task]:

1. `await_transform` ([task.promise] paragraph 9) routes the child through `as_awaitable`.
2. The awaiter calls `connect` ([task.members] paragraph 3-4), which creates a `state<Rcvr>` object.
3. The `state` constructor ([task.state] paragraph 2) stores the handle and receiver, constructs `own-env` from `get_env(rcvr)`, and constructs `Environment` from `own-env`.
4. `state::start` ([task.state] paragraph 4.3) initializes `SCHED(prom)` with `scheduler_type(get_scheduler(get_env(rcvr)))` - the scheduler must be extracted from the receiver's environment.
5. `state::start` ([task.state] paragraph 4.4-4.6) sets up stop token bridging: `prom.token` and `prom.source` must be initialized so that stop requests propagate from the parent.
6. `state::start` calls `handle.resume()`.

Steps 2-5 are normative requirements. No conforming implementation can skip them. The `state<Rcvr>` object, the scheduler extraction, and the stop token setup exist because the sender protocol requires `connect` / `start` to establish the execution context for the child.

In the coroutine-native model, the execution context is established by passing an `io_env` pointer - one store. The `io_env` already carries the executor, stop token, and frame allocator. No construction, no extraction, no bridging.

Per `co_await` of a child task. Multiplied by N in a chain of N coroutines.

The `state<Rcvr>` construction, scheduler extraction, and stop token setup are spec-mandated and remain in any conforming implementation.

5.4 Completion Cost

In the coroutine-native model, when a child task completes, `final_suspend` returns the parent's coroutine handle. Symmetric transfer. One pointer load.

In the sender model, `final_suspend` ([task.promise] paragraph 6) must invoke `set_value`, `set_error`, or `set_stopped` on `RCVR(*this)`. The receiver is the parent's awaiter. In the best case, the awaiter resumes the parent via symmetric transfer.

[task.promise] paragraph 9 specifies that `await_transform` skips `affine_on` when the sender type is the same `task` type. For the common I/O case - a chain of `task<T, IoEnv>` coroutines all running on the same executor - no scheduler comparison is needed on the completion path.

The completion path cost is granted as a concession.

5.5 The Combinator Gap

A purported benefit of the unified sender model is that combinator algorithms like `when_all` need only be written once. One `when_all` serves all domains - GPU tasks, I/O tasks, timers. The coroutine-native model needs a second implementation. This is a real argument. This section examines whether it holds for I/O.

5.5.1 The `when_all` Completion Handler

P2300R10[14] specifies `when_all`'s completion logic in `impls-for<when_all_t>::complete`. The handler dispatches on the completion channel tag:

```
[]<class Index, class State, class Rcvr,
    class Set, class... Args>(
    this auto& complete,
    Index, State& state, Rcvr& rcvr,
    Set, Args&&... args) noexcept
    -> void
{
    if constexpr (
        same_as<Set, set_error_t>)
    {
        if (disposition::error !=
            state.disp.exchange(
                disposition::error))
        {
            state.stop_src.request_stop();
            TRY-EMPLACE-ERROR(state.errors,
                forward<Args>(args)...);
        }
    }
    else if constexpr (
        same_as<Set, set_stopped_t>)
    {
        auto expected =
            disposition::started;
        if (state.disp
                .compare_exchange_strong(
                    expected,
                    disposition::stopped))
        {
            state.stop_src.request_stop();
        }
    }
    else if constexpr (
        !same_as<decltype(State::values),
                 tuple<>>)
    {
        if (state.disp ==
            disposition::started)
        {
            auto& opt =
                get<Index::value>(
                    state.values);
            TRY-EMPLACE-VALUE(complete,
                opt,
                forward<Args>(args)...);
        }
```

```
        }

        state.arrive(rcvr);
    }
```

Three branches. Three behaviors:

- `set_error` : calls `stop_src.request_stop()` . Siblings are cancelled. The error is stored.
- `set_stopped` : calls `stop_src.request_stop()` . Siblings are cancelled.
- `set_value` (the `else` branch): stores the values. No inspection. No stop request. Siblings keep running.

The `set_value` branch unconditionally stores values and calls `state.arrive(rcvr)` . There is no mechanism in the specification to inspect value-channel arguments, apply a predicate, or decide to cancel based on the values received.

5.5.2 Routing Options

An I/O read returns `(error_code, size_t)` . Kohlhoff identified the routing problem in 2021:

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel." (P2430R0[15])*

A fourth routing option resolves this: `set_error(tuple(ec, T...))` on failure, `set_value(T...)` on success. The full compound result - including the byte count - goes into the error channel as a tuple. The standard `when_all` sees `set_error` , cancels siblings, and stores the error. Both values are preserved inside the tuple.

Routing	Preserves	Breaks
`set_value(ec, n)` - value channel	Both values	`when_all` blind to errors
`set_error(ec)` - error channel	Composition algebra	Byte count destroyed
`set_value(pair{ec, n})` - value channel	Both values	Same as row 1
`set_error(tuple(ec, n))` - error channel	Both values	–

With the fourth routing, the standard `when_all` works correctly for I/O. The "write it once" argument holds. This paper grants this routing as a concession.

5.5.3 The Remaining Difference

The combinator gap is ergonomic, not structural. Both models can express the same semantics. The coroutine-native model provides a more convenient return type - `io_result<R1, ..., Rn>` with a single `ec` and flat destructuring - while the sender model returns `tuple<R1, ..., Rn>` on the value path and `set_error(tuple(ec, T...))` on the error path. The information is the same; the packaging differs.

The coroutine-native model's runner coroutine has direct access to the result value after `co_await`:

```
auto result =
    co_await std::move(inner);

std::get<Index>(
    state->results_).set(result);

if (result.ec)
    state->core_
        .stop_source_.request_stop();
```

The sender model achieves the same cancellation through channel dispatch.

5.6 `io_env` Indirection

In the coroutine-native model, the promise carries `io_env` directly. The type-erased executor is constructed once at task start. Every I/O operation receives a pointer to it. No indirection.

[exec.task] does not mandate virtual dispatch for environment access. The `state<Rcvr>` object stores `Environment` directly ([task.state] paragraph 2). A conforming implementation can provide direct access without indirection. This paper grants that the best-case implementation does so.

5.7 Stop Token Propagation

Cancellation in `when_all` requires propagating a stop signal from the shared stop source to each child's I/O operations. Both models create a stop source and bridge the parent's stop token to it.

Coroutine-native model (6 steps):

1. `when_all_core` creates a `stop_source_`.
2. A `stop_callback` bridges the parent's stop token to the shared stop source.
3. `launch_one` passes `stop_source_.get_token()` in `io_env` to each child.
4. The child's I/O awaitable reads `env->stop_token` directly.

5. On error, the runner calls `stop_source_.request_stop()`.
6. Siblings see `stop_requested()` in their `io_env.stop_token`.

One stop source. One parent bridge. One hop from stop source to child. The `io_env` carries the token directly.

Sender model (best case, per P2300R10[14] and [exec.task]):

1. [exec.when.all] paragraph 13: `when_all` creates an `inplace_stop_source` in its state object.
2. [exec.when.all] paragraph 18: `start` registers a stop callback bridging the parent receiver's stop token to the `inplace_stop_source`.
3. [exec.when.all] paragraph 12: each child receives an environment where `get_stop_token` returns the `inplace_stop_source`'s token.
4. [exec.when.all] paragraph 19: `when_all` calls `start` on each child's operation state.
5. [task.state] paragraph 4: `state::start` initializes `prom.token` and `prom.source` such that stop requests from the receiver's token propagate to the child's token ([task.state] paragraph 4.4-4.6). This is a per-child stop bridge.
6. [task.promise] paragraph 16.3: `get_env` returns `prom.token` via `get_stop_token`.
7. The child's I/O awaitable reads the stop token from the promise environment.
8. On error: [exec.when.all] paragraph 19 calls `stop_src.request_stop()`.
9. The stop request propagates through step 2's bridge to the `inplace_stop_source`.
10. The `inplace_stop_source` propagates through step 5's per-child bridges to each child's `prom.source`.
11. The I/O awaitable sees `stop_requested()`.

Property	Coroutine-native	Sender (best case)
Stop sources	1	1 + 1 per child
Parent stop bridge	1	1
Per-child stop bridges	0	1 per child
Steps to propagate stop	6	11
Spec reference	–	[task.state] paragraph 4

The coroutine-native model passes the shared stop token directly in `io_env` - one hop. The sender model must bridge the token through `state::start` for each child because [task.state] paragraph 4.4-4.6 requires `prom.token` and `prom.source` to be initialized per-child so that stop requests propagate. This is a normative requirement of [exec.task].

6. The Shipping Schedule Risk

The concessions in Section 2 assume six engineering fixes ship. P3552R3[1] is the vehicle. The C++26 cycle is closing. The fixes are not in P3552R3[1] today:

- Symmetric transfer: P3801R0[7] identified the vulnerability. Trampolines are being explored (P3796R1[11]). Not landed.
- Frame allocator timing: D3980R0[8] is in progress. Not landed.
- `IoEnv` : does not exist in any proposal.
- Error delivery: `AS-EXCEPT-PTR` is still in the specification.

If `task` ships in C++26 without these fixes, the concessions in Section 2 are hypothetical. The gap in Section 4 would be larger - the first seven rows would no longer say "Identical."

P2762R0[12] mentioned `io_task` in one paragraph:

> *"It may be useful to have a coroutine task (`io_task`) injecting a scheduler into asynchronous networking operations used within a coroutine... The corresponding task class probably needs to be templatized on the relevant scheduler type."*

P2762[13] stopped at R2 (October 2023). No revision in over two years. No published paper defines what `IoEnv` looks like inside `std::execution::task` .

The implementation section of P3552R3[1] acknowledges:

> *"This implementation hasn't received much use, yet, as it is fairly new."*

---

7. The Zero-Overhead Principle

P3406R0[17] (Stroustrup, 2024) states:

> *"Does the C++ design follow the zero-overhead principle? Should it? I think it should, even if that principle isn't trivial to define precisely. Some of you (for some definition of 'you') seem not to. We - WG21 as an organization - haven't taken it seriously enough to make it a requirement for acceptance of new features. I think that is a serious problem."*

The principle has two parts. P0709R4[18] (Sutter, 2019) defines them:

> *"'Zero overhead' is not claiming zero cost - of course using something always incurs some cost. Rather, C++'s zero-overhead principle has always meant that (a) 'if you don't use it you don't pay for it' and (b) 'if you do use it you can't reasonably write it more efficiently by hand.'"*

Part (b) applies here. The user is using a task abstraction for I/O. A coroutine-native task performs the same I/O without `state<Rcvr>` construction, without scheduler extraction, and without per-child stop token bridging. The overhead documented in Sections 4 and 5 exists because of the sender protocol, not because of the I/O operation.

P3406R0[17] continues:

> *"Every proposal, language and library, should be accompanied by a written discussion, ideally backed by measurements for credibility, to demonstrate that in the likely most common usage will not impose overheads compared to current and likely alternatives. Also, to demonstrate that optimizations and tuning will not be inhibited. If no numbers are available or possible, the committee should be extremely suspicious and in particular, never just assume that optimizations will emerge over time. This should be a formal requirement."*

This paper is that written discussion. The measurements are in Sections 4 and 5. The current and likely alternative is the coroutine-native task. The overheads are spec-mandated and cannot be assumed away by future optimizations.

---

8. Conclusion

This paper grants every proposed fix and compares against the best possible conforming implementation of `std::execution::task`.

Per [exec.task], every `co_await` of a child task must construct a `state<Rcvr>` object, extract the scheduler from the receiver's environment, and set up stop token bridging. In the coroutine-native model, the same path stores two pointers. The sender protocol requires `connect` / `start` to establish the execution context per-child. The coroutine-native model passes an `io_env` pointer because the execution context is propagated, not reconstructed.

The gap is spec-mandated and cannot be optimized away by a better implementation. A typical I/O session - accept, authenticate, read request, process, write response - is a chain of roughly 5 coroutines. The per-session cost:

Operation	Coroutine-native	Best `task<T, IoEnv>`
`state<Rcvr>` constructions	0	5
Scheduler extractions	0	5
Stop token bridges	0	5
Pointer stores	10	10

Multiplied by the number of concurrent sessions. The coroutine-native model pays 10 pointer stores. The sender model pays the same 10 pointer stores plus 15 additional operations mandated by [exec.task].

The committee has the information to evaluate whether the gap justifies a separate task type for I/O. This paper provides the evidence. The committee decides.

Acknowledgments

The author thanks Bjarne Stroustrup for P3406R0 and for articulating the standard to which this paper holds itself; Herb Sutter for the two-part definition of zero overhead in P0709R4; Dietmar Kühl for P3552R3 and P3796R1 and for `beman::execution`; Jonathan Müller for P3801R0 and for documenting the symmetric transfer gap; Chris Kohlhoff for identifying the partial-success problem in P2430R0; Eric Niebler, Lewis Baker, and Kirk Shoop for `std::execution`; Steve Gerbino for co-developing the bridge implementations; Peter Dimov for the frame allocator propagation analysis and for the `set_error(tuple(ec, T...))` routing that resolves the combinator gap; and Mungo Gill, Mohammad Nejati, Michael Vandeberg, and Andrzej Krzemieński for feedback.

References

1. P3552R3 - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
2. bemanproject/task - P3552R3 reference implementation. https://github.com/bemanproject/task
3. D4051R0 - "Steelmanning P3552R3" (Vinnie Falco, 2026). https://wg21.link/d4051r0
4. cppalliance/corosio - Coroutine-native networking library. https://github.com/cppalliance/corosio

5. cppalliance/capy - Coroutine I/O primitives library. https://github.com/cppalliance/capy

6. P4088R0 - "The Case for Coroutines" (Vinnie Falco, 2026). https://isocpp.org/files/papers/P4088R0.pdf

7. P3801R0 - "Concerns about the design of `std::execution::task`" (Jonathan Müller, 2025). https://wg21.link/p3801r0

8. D3980R0 - "Task's Allocator Use" (Dietmar Kühl, 2026). https://isocpp.org/files/papers/D3980R0.html

9. P4092R0 - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://isocpp.org/files/papers/P4092R0.pdf

10. P4093R0 - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://isocpp.org/files/papers/P4093R0.pdf

11. P3796R1 - "Coroutine Task Issues" (Dietmar Kühl, 2025). https://wg21.link/p3796r1

12. P2762R0 - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). https://wg21.link/p2762r0

13. P2762R2 - "Sender/Receiver Interface For Networking" (Dietmar Kühl, 2023). https://wg21.link/p2762r2

14. P2300R10 - "std::execution" (Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach, 2024). https://wg21.link/p2300r10

15. P2430R0 - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). https://wg21.link/p2430r0

16. P4003R0 - "Coroutine-Native I/O" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4003r0

17. P3406R0 - "We need better performance testing" (Bjarne Stroustrup, 2024). https://wg21.link/p3406r0

18. P0709R4 - "Zero-overhead deterministic exceptions: Throwing values" (Herb Sutter, 2019). https://wg21.link/p0709r4